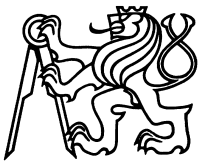
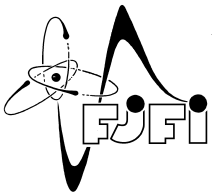


Czech Technical University in Prague
Faculty of Nuclear Sciences and Physical Engineering



Automatic Monitoring of Data Drift and Model Performance

Automatický monitoring driftu dat a výkonu modelu

Bachelor's Degree Project

Author: **Bexultan Zhaskairat**
Supervisor: **RNDr. Zuzana Petříčková, Ph.D.**
Consultant: **doc. RNDr. Jméno Konzultanta, CSc.**
Academic year: 2025/2026

January 20, 2025

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Zhaskairat** Jméno: **Bexultan** Osobní číslo: **527893**

Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**

Zadávající katedra/ústav: **Katedra matematiky**

Studijní program: **Aplikovaná informatika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Automatický monitoring driftu dat a výkonu modelu

Název bakalářské práce anglicky:

Automatic Monitoring of Data Drift and Model Performance

Jméno a pracoviště vedoucí(ho) bakalářské práce:

RNDr. Zuzana Petříčková, Ph.D. katedra softwarového inženýrství FJFI

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **31.10.2025** Termín odevzdání bakalářské práce: **03.08.2026**

Platnost zadání bakalářské práce: **30.09.2027**

Zuzana Masáková
Digitálně podepsal(a) Zuzana Masáková
Datum: 07.11.2025 13:48:11
ID: 40553

podpis vedoucí(ho) ústavu/katedry

Milan Krbálek
Digitálně podepsal(a) Milan Krbálek
Datum: 08.11.2025 08:05:12
ID: 40556

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

08.11.2025
Datum převzetí zadání

Zhaskairat Bexultan
Podpis studenta

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Zhaskairat** Jméno: **Bexultan** Osobní číslo: **527893**

Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**

Zadávající katedra/ústav: **Katedra matematiky**

Studijní program: **Aplikovaná informatika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Automatický monitoring driftu dat a výkonu modelu

Název bakalářské práce anglicky:

Automatic Monitoring of Data Drift and Model Performance

Pokyny pro vypracování:

1. Seznamte se s principy MLOps a problematikou provozního monitoringu modelů strojového učení (data drift, concept drift, kalibrace, SLA/SLO).
2. Zpracujte přehled metod detekce driftu (např. statistické testy a streamové detektory) a charakterizujte jejich vlastnosti, výhody a omezení.
3. Definujte požadavky na vytvářený monitorovací systém, navrhněte jeho architekturu, vyberte vhodné metody pro detekci driftu a integrujte je do systému. Implementujte funkční prototyp.
4. Získejte nebo připravte dataset vhodný pro demonstraci driftu dat. Vytvořte testovací scénáře s řízenými změnami.
5. Ověřte a srovnejte chování a výkon implementovaných metod. Analyzujte vliv hlavních parametrů a vyhodnoťte přesnost i včasnost detekce driftu. Na závěr zhodnoťte celkovou funkčnost a přínos navrženého řešení.

Seznam doporučené literatury:

[1] GAMA, João; ŽLIOBAITĖ, Indrė; BIFET, Albert; PECHENIZKIY, Mykola; BOUCHERON, Abdelhamid. A Survey on Concept Drift Adaptation. ACM Computing Surveys, 2014.
[2] LU, Jie; LIU, Anjin; DONG, Fan; GU, João; WANG, Guangquan. Learning under Concept Drift: A Review. IEEE TKDE, 2019.
[3] GRETTON, Arthur; et al. A Kernel Two-Sample Test. Journal of Machine Learning Research, 2012.
[4] BIFET, Albert; GAVALDÀ, Ricard. ADWIN: Adaptive Windowing. SDM, 2007.
[5] GUO, Chuan; PLEISS, Geoff; SUN, Yu; WEINBERGER, Kilian Q. On Calibration of Modern Neural Networks. ICML, 2017.
[6] Evidently AI Documentation. https://docs.evidentlyai.com

FAKULTA JADERNÁ A FYZIKÁLNĚ INŽENÝRSKÁ
FACULTY OF NUCLEAR SCIENCES AND PHYSICAL ENGINEERING
Břehová 7
115 19 Praha 1

DECLARATION

I, the undersigned

Student's surname, given name(s):	Zhaskairat Bexultan
Personal number:	527893
Programme name:	Applied Informatics

declare that I have elaborated the bachelor's thesis entitled

Automatic Monitoring of Data Drift and Model Performance

independently, and have cited all information sources used in accordance with the Methodological Instruction on the Observance of Ethical Principles in the Preparation of University Theses and with the Framework Rules for the Use of Artificial Intelligence at CTU for Academic and Pedagogical Purposes in Bachelor's and Continuing Master's Programmes.

I declare that I used artificial intelligence tools during the preparation and writing of this thesis. I verified the generated content. I hereby confirm that I am aware of the fact that I am fully responsible for the contents of the thesis.

In Prague on 31.01.2026

Bexultan Zhaskairat
.....
student's signature

Acknowledgments:

I would like to thank . . . for . . . and express my gratitude to . . . for . . .

Bexultan Zhaskairat

Název práce:
Automatický monitoring driftu dat a výkonu modelu

Autor: Bexultan Zhaskairat

Studijní program: Aplikovaná informatika

Druh práce: Bakalářská práce

Vedoucí práce: RNDr. Zuzana Petříčková, Ph.D.

Abstrakt:

Klíčová slova: automatický monitoring, concept drift, data drift, detekce anomálií, MLOps, produkční systémy, strojové učení, výkonnost modelu.

Title:

Automatic Monitoring of Data Drift and Model Performance

Author: Bexultan Zhaskairat

Abstract:

Key words: anomaly detection, concept drift, data drift, machine learning, MLOps, model monitoring, model performance, production systems.

Contents

Introduction

Machine learning models are widely used in real-world applications to support decision-making and automate business processes. However, their performance can decrease after deployment because production data may change over time due to shifts in user behavior, external conditions, business processes, missing data, application errors, schema changes, or changes in data sources [1, 2]. This problem is closely related to data drift, where the data used in production becomes different from the data used during training or validation [1].

Because of this, model monitoring is an important part of the machine learning lifecycle. It helps observe whether a deployed model continues to behave as expected in production. Monitoring can include tracking prediction quality, error rates, confidence scores, latency, feature distributions, missing values, and schema changes [1].

A reliable monitoring process should also focus on input data quality. Important indicators include data volume, average values of input variables, and missing or null values [1]. In addition, statistical methods such as Population Stability Index, Kolmogorov–Smirnov test, Wasserstein distance, and Chi-square test can be used to compare reference data with current data [3, 4]. If significant differences are detected, alerts can notify developers or data scientists that the model may no longer be performing reliably.

The aim of this bachelor thesis is to design and implement a prototype system for automatic monitoring of data drift and model performance. The project focuses on operational monitoring of machine learning models, data quality checks, batch and streaming drift detection, model performance evaluation, result storage, and dashboard visualization.

Chapter 1

Operational Monitoring of Machine Learning Models

1.1 Machine Learning Models in Production

Machine learning models are usually trained on historical data and later applied to new data in a production environment. In contrast to traditional software, the behavior of an ML model depends not only on source code, but also on the statistical properties of the data used during inference. Therefore, a deployed model can become less reliable even when the application code remains unchanged.

Production model performance can degrade because of changes in the external environment, missing input features, schema changes, data quality problems, seasonality, or changes in user behavior [1]. This means that a model which performed well during training and validation may later produce weaker predictions on current production data.

For this reason, monitoring is an important part of the MLOps lifecycle. ML implementation is commonly described as an end-to-end process that includes data handling, model building, validation, deployment, continuous monitoring, performance tracking, and maintenance [2]. In this thesis, the focus is not placed on building a complete industrial MLOps platform, but on implementing the monitoring part of this lifecycle.

A production ML system should therefore be observed not only as software, but also as a data-driven decision system. Traditional monitoring can detect technical failures such as latency or service errors, while ML monitoring must also detect changes in data, predictions, model performance, and business impact [5].

1.2 Data Quality and ML-Specific Monitoring

Standard software monitoring usually focuses on technical indicators such as uptime, latency, memory usage, error rates, or resource consumption. These indicators are important, but they do not show whether an ML model still produces reliable predictions. ML-specific monitoring extends this view by observing the quality of input data, the distribution of model predictions, and the predictive performance of the model.

Data quality is important because poor input data can strongly affect model behavior. Production data may contain missing values, outliers, inconsistent records, different data formats, or changed schemas. These problems can appear because of application bugs, upstream pipeline changes, source system errors,

or changes in business processes [1]. For this reason, data validation and data quality checks are necessary before drift detection and performance monitoring are interpreted.

A practical ML monitoring process can include checks such as data volume, missing value rate, feature ranges, average feature values, type consistency, and schema compatibility [1]. These checks do not directly prove that a model performs poorly, but they can reveal problems that may cause unreliable predictions.

In this thesis, ML-specific monitoring consists of three main parts:

- **Data quality monitoring:** checking whether uploaded datasets have a usable and comparable structure.
- **Data drift monitoring:** comparing feature distributions between reference and current datasets.
- **Model performance monitoring:** evaluating prediction quality when a model and labeled data are available.

This separation is useful because each type of monitoring answers a different question. Data quality monitoring checks whether the data is valid. Drift monitoring checks whether the current data is different from the reference data. Performance monitoring checks whether the model still produces acceptable predictions.

1.3 Model Performance Degradation and Calibration

Model performance degradation means that the predictive quality of a deployed model becomes worse over time. This can happen when input data changes, when the relationship between features and the target changes, or when data quality problems appear in the production pipeline [1]. A performance drop can lead to incorrect predictions and poor decisions, especially when the model is used in business-critical processes.

Performance degradation can be measured directly only when true labels are available. For classification tasks, common metrics include accuracy, precision, recall, F1-score, and AUC-ROC. For regression tasks, metrics such as MAE, MSE, RMSE, and MAPE can be used [6]. The selected metric must match the goal of the model. For example, recall may be more important in fraud detection, while precision may be more important when false alarms are costly.

Model calibration is also relevant for classification models that output probabilities. A model is well calibrated when predicted probabilities correspond to real outcome frequencies. For example, if a model predicts a probability of 0.8 for a group of cases, approximately 80% of those cases should belong to the positive class. Poor calibration can make a model unreliable even when its classification accuracy appears acceptable. Calibration can be evaluated using reliability diagrams, which compare predicted confidence with observed accuracy, and by metrics such as Expected Calibration Error (ECE), which calculates the weighted difference between average confidence and average accuracy across confidence bins [7].

In this thesis, calibration is treated as part of the theoretical background. The prototype mainly focuses on drift detection and model performance metrics. However, calibration can be considered as a future extension, especially for classification models where probability outputs are used for decision-making.

1.4 Monitoring Indicators: SLA, SLO, and SLI

Monitoring results must be converted into clear and understandable indicators. In operational systems, this is often described through service-level concepts: SLA, SLO, and SLI.

A **Service Level Indicator** (SLI) is a measurable value that describes the behavior of a system. In ML monitoring, examples of SLIs include missing value rate, number of drifted features, PSI value, prediction distribution shift, accuracy, F1-score, MAE, or inference latency.

A **Service Level Objective** (SLO) defines the expected or acceptable level for an indicator. For example, an SLO can state that model accuracy should remain above a selected threshold, that the percentage of missing values should stay below a defined limit, or that PSI should not exceed a selected drift threshold.

A **Service Level Agreement** (SLA) is a formal agreement about expected service quality. In a bachelor thesis prototype, SLA is mostly theoretical because the system is not deployed as a commercial service. However, the idea is still useful because it shows how monitoring thresholds can be connected to operational expectations.[8]

In this prototype, we use monitoring indicators mainly as technical and analytical thresholds. Drift scores, data quality checks, and model performance metrics are used to decide whether the monitoring run should be interpreted as stable, warning-level, or degraded.

Chapter 2

Data Drift and Drift Detection Methods

2.1 Taxonomy of Dataset Shift

Dataset shift describes a situation where the data observed after deployment differs from the data used during model development. This is a central problem in production ML because models are usually trained under the assumption that future data will be similar to training or validation data [1]. When this assumption is violated, model predictions may become less reliable.

2.1.1 Types of Dataset Shift

The different types of dataset shift can be described using the joint distribution of input features X and target variable Y. This distribution can be decomposed as

$$P(X, Y) = P(Y \mid X)P(X), \tag{2.1}$$

or equivalently as

$$P(X, Y) = P(X \mid Y)P(Y). \tag{2.2}$$

Based on this decomposition, different forms of shift can be understood as changes in different parts of the data-generating process. Data drift is mainly connected with a change in $P(X)$, concept drift with a change in $P(Y \mid X)$, and prior probability shift with a change in $P(Y)$ while $P(X \mid Y)$ remains stable [9, 10].

Data drift, also called covariate drift, occurs when the distribution of input features changes over time. In this case, the model receives data that differs from the data used during training, validation, or an earlier stable production period [1]. Data drift can appear as changed numerical values, changed categorical frequencies, new categories, or changes in missing values.

Concept drift occurs when the relationship between input features and the target variable changes. In this case, similar input values may require different predictions than before. Concept drift is usually more difficult to detect because it often requires ground truth labels or performance-based signals [11].

Prediction drift occurs when the distribution of model outputs changes. Even if input features appear stable, a significant change in prediction distribution may indicate a model behavior change, a data pipeline issue, or a change in the population being scored [5].

Prior probability shift / label shift occurs when the distribution of the target variable changes, while the conditional distribution of features given the class remains stable. Formally, this means that $P(Y)$ changes but $P(X \mid Y)$ stays the same. In practice, this can happen when the proportion of classes changes over time, for example when one customer group, risk class, or event type becomes more common than before [9].

These types of drift are related but not identical. Data drift can happen before performance degradation becomes visible. Concept drift can directly reduce model quality. Prediction drift can work as an early warning signal when labels are delayed or unavailable. Pure covariate shift does not necessarily reduce model quality by itself, which is why drift indicators and model performance metrics are monitored separately [9].

2.1.2 Types and Causes of Drift

Drift can appear in different forms. It can be sudden, gradual, incremental, or recurring [11]. Sudden drift appears quickly, for example after a major external event or a system change. Gradual drift appears slowly as a new pattern replaces an older one over time. Incremental drift describes a continuous transition from one concept to another. Recurring drift appears when an older pattern returns, for example because of seasonality.

The causes of drift can be internal or external. Internal causes include schema changes, broken data pipelines, missing features, changed preprocessing logic, or application bugs. External causes include user behavior changes, seasonality, market changes, economic conditions, new competitors, or changes in the environment where the model is used [1].

For this thesis, the most important practical case is batch data drift between a reference dataset and a current dataset. The prototype does not focus on full streaming adaptation, but on detecting whether the current uploaded data differs from the selected baseline.

2.2 Reference Data and Current Data

The monitoring approach in this thesis is based on comparing reference data and current data.

The **reference data** is the baseline dataset. It can come from training data, validation data, or a previously stable production period. It represents the expected structure and distribution of the data.

The **current data** is the new dataset that should be checked. In the prototype, we upload or select a current dataset and compare it with the reference dataset.

This distinction is necessary because drift detection requires a comparison point. Without reference data, the system cannot determine whether the current data is normal or shifted. A similar monitoring approach creates benchmark distribution statistics from training and validation data and later compares them with production distribution statistics [1].

In the implemented prototype, we use this reference-current comparison as the central monitoring workflow. The system compares feature distributions, calculates drift indicators, and stores the results as monitoring runs.

2.3 Drift Detection Methods

Drift detection methods are used to determine whether data has changed enough to require attention. Different methods are suitable for different types of data and monitoring scenarios. In general, drift detection methods can be grouped into statistical tests, distance-based methods, and stream detectors [11].

2.3.1 Statistical and Distance-Based Drift Metrics

This thesis uses selected common methods for detecting data drift between a reference dataset and a current dataset: the Kolmogorov–Smirnov test, Population Stability Index, and Wasserstein distance. These methods are suitable for batch comparison because they compare two observed data distributions

and return either a statistical test result or a drift score. The selected methods are also commonly used in practical ML monitoring workflows for numerical feature drift detection [3, 4]. This feature-by-feature testing approach must be interpreted carefully because testing many features increases the chance of false positive drift detections. This can be handled either by applying a multiple-testing correction, such as the Bonferroni correction, or by using a dataset-level aggregation rule, for example the share of features marked as drifted.

Kolmogorov–Smirnov Test

The Kolmogorov–Smirnov (KS) test is a non-parametric statistical test used to compare two numerical distributions. It does not assume that the data follows a normal distribution. The null hypothesis states that the reference and current samples come from the same distribution. If the resulting p-value is below a selected significance level, the feature can be marked as drifted [3, 4]. The KS statistic is defined as:

$$D = \sup_x |F_{\text{ref}}(x) - F_{\text{cur}}(x)|, \tag{2.3}$$

where $F_{\text{ref}}(x)$ is the empirical cumulative distribution function of the reference data, $F_{\text{cur}}(x)$ is the empirical cumulative distribution function of the current data, and $\sup_x$ denotes the maximum difference over all values of x.

The advantage of the KS test is its high sensitivity. It can detect even small changes in numerical distributions. However, this can also be a disadvantage for large datasets, because small but practically insignificant differences may become statistically significant. Therefore, when large datasets are used, sampling or stricter significance thresholds may be needed [3].

Population Stability Index

Population Stability Index (PSI) measures how much a variable distribution has changed between two samples or time periods. It is commonly used in monitoring tasks where the goal is to detect meaningful population changes rather than very small statistical differences.

PSI is calculated as:

$$\text{PSI} = \sum_{i=1}^{n} (r_i - c_i) \ln\left(\frac{r_i}{c_i}\right), \tag{2.4}$$

where r_i is the proportion of observations in bin i in the reference dataset, c_i is the proportion of observations in bin i in the current dataset, and n is the number of bins.

A common interpretation is:

- **PSI** < 0.1**:** no significant population change;
- $0.1 \leq$ **PSI** < 0.2**:** moderate population change;
- **PSI** ≥ 0.2**:** significant population change.

Compared with the KS test, PSI is less sensitive to small changes and is more predictable for large datasets. This makes it useful when the monitoring system should react only to larger and more interpretable distribution changes [3].

Wasserstein Distance

Wasserstein distance, also known as Earth-Mover Distance, measures how much effort is needed to transform one distribution into another. For numerical features, it can be interpreted as the amount of movement required to align the current distribution with the reference distribution [3, 4].

For one-dimensional distributions, the 1st Wasserstein distance can be written as:

$$W(P, Q) = \int_{-\infty}^{\infty} \left| F_P(x) - F_Q(x) \right| \, dx, \tag{2.5}$$

where $F_P(x)$ and $F_Q(x)$ are the cumulative distribution functions of the compared distributions.

Unlike PSI, Wasserstein distance works directly with numerical values and does not necessarily require binning. It is useful when the magnitude of numerical change matters. However, because the value depends on the scale of the feature, it may be difficult to compare across features with different units. A normalized version can be used by dividing the distance by the standard deviation:

$$W_{\text{norm}}(P, Q) = \frac{W(P, Q)}{\sigma_{\text{ref}}}, \tag{2.6}$$

where σ_{ref} is the standard deviation of the reference feature. This makes the result easier to compare across numerical features. In practical monitoring, Wasserstein distance can provide a compromise between the high sensitivity of the KS test and the lower sensitivity of PSI[3].

These methods are suitable for the prototype because the system works mainly with uploaded reference and current datasets. Therefore, the task is closer to batch comparison than continuous stream monitoring.

Chi-Square Test

The Chi-square test is commonly used for categorical features. It compares the observed category frequencies in the current dataset with the expected frequencies from the reference dataset. The test statistic is calculated as:

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}, \tag{2.7}$$

where O_i is the observed frequency of category i, E_i is the expected frequency of category i, and k is the number of categories. If the p-value is below the selected significance level, the categorical feature can be marked as drifted [4].

Anderson–Darling Test

The Anderson–Darling test is another statistical test for comparing distributions. Compared with the Kolmogorov–Smirnov test, it gives more weight to differences in the tails of the distribution. This can be useful when rare but important values change between the reference and current datasets.

A simplified two-sample form can be written as:

$$A^2 = \int_{-\infty}^{\infty} \frac{(F_{\text{ref}}(x) - F_{\text{cur}}(x))^2}{H(x)(1 - H(x))} \, dH(x), \tag{2.8}$$

where $F_{\text{ref}}(x)$ and $F_{\text{cur}}(x)$ are empirical cumulative distribution functions, and $H(x)$ is the pooled empirical distribution. A larger value indicates a stronger difference between the compared distributions [4].

Cramér–von Mises Test

The Cramér–von Mises test compares two empirical distributions by measuring the squared difference between their cumulative distribution functions. Unlike the Kolmogorov–Smirnov test, which focuses on the maximum difference, this method considers differences across the whole distribution.

The statistic can be written as:

$$\omega^2 = \int_{-\infty}^{\infty} (F_{\text{ref}}(x) - F_{\text{cur}}(x))^2 \, dH(x), \tag{2.9}$$

where $F_{\text{ref}}(x)$ and $F_{\text{cur}}(x)$ are the empirical cumulative distribution functions of the compared datasets, and $H(x)$ is the pooled empirical distribution. A larger value means that the two distributions differ more strongly [4].

2.3.2 Stream Detectors

Stream detectors are used when data arrives continuously. Instead of comparing one reference dataset with one current dataset, they process observations over time and try to detect the moment when the data-generating process changes.

In this thesis, four stream detectors are discussed: Drift Detection Method (DDM), Adaptive Windowing (ADWIN), Early Drift Detection Method (EDDM), and Page-Hinkley Test. DDM is used as the foundational baseline because it represents the classical error-rate monitoring approach. ADWIN is treated as the main streaming method because it is more flexible: it uses an adaptive window and can resize it automatically when the stream changes [12, 13, 14].

Drift Detection Method

Drift Detection Method (DDM) monitors the error rate of an online classifier. Under a stable distribution, the error rate should decrease as more instances are processed. If the error rate increases significantly, DDM raises either a warning or a drift signal [12].

At time step i, DDM tracks the error rate p_i and standard deviation s_i:

$$s_i = \sqrt{\frac{p_i(1 - p_i)}{i}}. \tag{2.10}$$

The method stores the minimum observed value $p_{\min} + s_{\min}$. A warning is raised when:

$$p_i + s_i \geq p_{\min} + 2s_{\min}, \tag{2.11}$$

and drift is detected when:

$$p_i + s_i \geq p_{\min} + 3s_{\min}. \tag{2.12}$$

DDM is simple and interpretable, which makes it useful as a baseline. Its main limitation is that it depends on labeled data because it must know whether each prediction was correct.

Adaptive Windowing

Adaptive Windowing (ADWIN) detects drift using a variable-length sliding window. When the stream is stable, the window grows. When a change is detected, older observations are removed and the detector focuses on newer data [13].

ADWIN splits the current window W into two subwindows, W_0 and W_1, and compares their means:

$$|\mu_{W_0} - \mu_{W_1}| > \epsilon. \tag{2.13}$$

If the difference is statistically significant, ADWIN reports drift. Its main advantage is that it does not require a fixed window size. This makes it more suitable for changing production streams than basic error-rate methods. The threshold ϵ is not a fixed hyperparameter, but is dynamically calculated using the Hoeffding bound, which guarantees a strict limit on the false positive rate depending on a user-defined confidence level δ.

In this thesis, DDM serves as the baseline stream detector, while ADWIN is used as the preferred adaptive method for the streaming part of the project. DDM provides a simple reference point based on error-rate monitoring, whereas ADWIN is applied to detect changes in continuous data streams with a dynamically adjusted window.

Early Drift Detection Method

Early Drift Detection Method (EDDM) is an extension of DDM. Instead of monitoring only the error rate, it monitors the distance between classification errors. Under stable conditions, the distance between errors should increase as the classifier learns from the stream. If the distance between errors starts to decrease, it may indicate that the model is making mistakes more frequently and that drift is occurring [14].

EDDM is especially useful for gradual drift because it can react to slower changes in the data stream. However, like DDM, it requires labeled data because it must know whether each prediction is correct. Therefore, it is mainly suitable for supervised online monitoring scenarios.

Page-Hinkley Test

The Page-Hinkley Test is a sequential change detection method used to identify changes in the average value of a monitored variable. In drift detection, it can be applied to prediction errors, loss values, or other streaming measurements. The method tracks cumulative deviations from the observed mean and raises a drift signal when the accumulated difference exceeds a selected threshold [14].

A simplified form of the monitored cumulative difference can be written as:

$$m_t = m_{t-1} + (x_t - \bar{x}_t - \delta), \tag{2.14}$$

where x_t is the observed value at time t, $\bar{x}_t$ is the running mean, and δ is a tolerance parameter for small changes. Drift is detected when the difference between the cumulative value and its minimum becomes larger than a predefined threshold.

The advantage of the Page-Hinkley Test is that it can detect changes in a continuous stream without storing all previous observations. Its limitation is that the result depends on the selected threshold and tolerance parameters.

2.3.3 Selected Metrics and Tools for the Prototype

The monitoring prototype uses different drift detection methods depending on the data type and monitoring mode. For batch comparison between a reference dataset and a current dataset, we use statistical and distance-based metrics. For streaming data, we use detectors that process observations over time.

For batch drift detection, the prototype focuses on three methods: the Kolmogorov–Smirnov test, Wasserstein distance, and Population Stability Index. These methods were selected because they represent different but complementary approaches to distribution comparison. The Kolmogorov–Smirnov test provides a statistical test with a p-value, Wasserstein distance measures the magnitude of numerical distribution change, and PSI gives an interpretable stability score based on binned distributions [3, 4].

The number of batch methods is intentionally limited. Adding many drift metrics would make the prototype more complex without necessarily improving the main goal of the thesis. Each additional method requires implementation, parameter selection, result interpretation, testing, and visualization. For a bachelor thesis prototype, this would increase the scope and make the system harder to evaluate clearly. Therefore, the selected three methods provide a practical balance between theoretical coverage, implementation effort, and interpretability.

For streaming drift detection, the prototype focuses on DDM and ADWIN. DDM is used as the baseline stream detector because it represents the classical error-rate monitoring approach. It tracks the model error rate and raises a warning or drift signal when the error increases significantly. ADWIN is used as the preferred adaptive stream detector because it maintains a variable-length window and automatically reduces the window when a statistically significant change is detected [12, 13, 14].

Other stream detectors, such as EDDM and the Page-Hinkley Test, are useful in concept drift research, but they are not implemented in the prototype. Including several stream detectors would require additional stream simulation, parameter tuning, comparison logic, and separate evaluation of detection delay and false alarms. This would make the project too heavy for the intended MVP. DDM and ADWIN are sufficient for the prototype because they allow a clear comparison between two important approaches: supervised error-rate monitoring and adaptive-window drift detection.

Model performance metrics are calculated when true labels are available. For classification tasks, the prototype uses accuracy, precision, recall, F1-score, and optionally AUC-ROC. For regression tasks, it uses MAE, MSE, RMSE, and MAPE. These metrics show whether the model still produces reliable predictions on current or streaming data [6].

The evaluation of monitoring results should consider both detection quality and practical usefulness. For batch drift detection, the system checks whether the selected metrics correctly identify controlled changes in the current dataset. For streaming drift detection, DDM and ADWIN can be evaluated using true detection rate, false detection rate, missed detection rate, and detection delay [11].

The prototype does not aim to replace a complete commercial ML monitoring platform. Instead, we implement the core monitoring logic directly in the application: dataset upload, reference-current comparison, streaming drift detection, drift metric calculation, performance metric calculation, result storage, and dashboard visualization. External tools such as SciPy, Evidently AI, and River support these workflows, while the thesis demonstrates the monitoring process in a transparent and reproducible MVP.

Multivariate drift detection methods, such as Maximum Mean Discrepancy or domain-classifier-based drift detection, as well as drift adaptation and automatic retraining strategies, are deliberately left out of scope and considered future work.

Chapter 3

System Requirements and Architecture

3.1 Purpose of the System

The purpose of the system is to provide a prototype for automatic monitoring of data drift and model performance. The system is designed to compare reference data with current data, detect distribution changes, evaluate model performance when true labels are available, and present the results in a dashboard.

The prototype follows the main goals of the thesis assignment: operational monitoring of machine learning models, detection of data drift, implementation of selected drift detection methods, preparation of controlled drift scenarios, and evaluation of monitoring results. [1]

The system is not intended to replace a full industrial MLOps platform. Instead, it demonstrates the core workflow of model monitoring in a controlled bachelor thesis prototype.

3.2 Functional Requirements

Functional requirements describe what the system must provide to the user. The main functional requirements are listed in Table 3.1.

ID	Requirement
FR1	The system must allow the user to upload or select a reference dataset.
FR2	The system must allow the user to upload or select a current dataset.
FR3	The system must validate whether the datasets can be compared.
FR4	The system must calculate drift metrics for selected features.
FR5	The system must support statistical and distance-based drift detection for batch data.
FR6	The system must support stream drift detection using DDM and ADWIN.
FR7	The system must allow the user to upload or register a machine learning model.
FR8	The system must calculate model performance metrics when true labels are available.
FR9	The system must store monitoring runs and calculated results.
FR10	The system must display drift results, model performance metrics, and run summaries in a dashboard.

Table 3.1: Functional requirements of the monitoring prototype

3.3 Non-Functional Requirements

Non-functional requirements describe the expected quality of the system. Since the project is a prototype, the focus is placed on clarity, reproducibility, and extensibility rather than large-scale production deployment.

Requirement	Description
Usability	The dashboard should present monitoring results in a clear and understandable way.
Reproducibility	Monitoring runs should be stored so that results can be reviewed later.
Modularity	Dataset handling, drift detection, model performance calculation, and visualization should be separated.
Extensibility	New drift detectors and performance metrics should be possible to add later.
Reliability	Invalid files, missing columns, and incompatible datasets should be handled with clear error messages.
Performance	The prototype should work with small and medium-sized datasets suitable for demonstration and evaluation.
Security	Uploaded files should be handled safely, especially model artifacts and user-provided datasets.

Table 3.2: Non-functional requirements of the monitoring prototype

3.4 System Architecture

The prototype follows a client-server architecture. The frontend provides the user interface for uploading data, starting monitoring runs, and viewing results. The backend handles dataset validation, drift calculation, model handling, performance evaluation, and result storage. The database stores metadata about datasets, models, monitoring runs, drift results, and performance results.

The architecture consists of the following main components:

- **Frontend dashboard:** provides interaction with the monitoring system and displays results.
- **Backend API:** exposes endpoints for datasets, drift runs, model artifacts, and performance monitoring.
- **Dataset service:** validates uploaded datasets and prepares them for comparison.
- **Drift detection service:** calculates batch and stream drift detection results.
- **Model performance service:** loads or registers models and calculates prediction metrics.
- **Database:** stores metadata, monitoring runs, drift results, and performance results.
- **File storage:** stores uploaded datasets and model artifacts.

This separation makes the system easier to maintain and extend. For example, additional drift detectors can be added to the drift detection service without changing the dashboard logic.

3.5 Data Flow

The system workflow starts with dataset selection or upload. The reference dataset represents the baseline distribution, while the current dataset represents new data that should be checked.

The batch monitoring workflow is as follows:

1. The user uploads or selects a reference dataset.
2. The user uploads or selects a current dataset.
3. The backend validates file format, columns, and basic data quality.
4. The drift detection service compares selected features.
5. The system calculates drift scores and drift status for each feature.
6. The results are stored as a monitoring run.
7. The dashboard displays the summary and feature-level results.

The model performance workflow is executed when a model and true labels are available:

1. The user selects or uploads a model artifact.
2. The system applies the model to the current dataset.
3. Predictions are compared with true labels.
4. Classification or regression metrics are calculated.
5. The performance results are stored and displayed in the dashboard.

For streaming drift detection, observations are processed over time. DDM monitors changes in classifier error rate, while ADWIN detects changes using an adaptive window. The output of the stream detector is stored as a drift signal and can be displayed as part of the monitoring results.

3.6 Database Design

The database is used to store metadata and monitoring results. The main entities are:

- **Dataset:** stores information about uploaded reference and current datasets.
- **Model artifact:** stores metadata about uploaded or registered models.
- **Monitoring run:** represents one execution of drift or performance monitoring.
- **Drift result:** stores feature-level drift scores and drift status.
- **Performance result:** stores calculated model performance metrics.

The database does not need to store all analytical logic. Its main purpose is to preserve results, support reproducibility, and allow the user to review previous monitoring runs.

3.7 Technology Stack

The prototype uses Python-based tools because the project focuses on machine learning monitoring and data analysis. The main technologies are:

- **Python:** core programming language for data processing and monitoring logic.
- **FastAPI:** backend API for dataset upload, drift runs, and model-related operations.
- **Pandas:** dataset loading, preprocessing, and feature-level analysis.
- **SciPy:** statistical tests and distance-based drift metrics.
- **River:** stream drift detection, especially DDM and ADWIN.
- **Scikit-learn:** model evaluation metrics and model compatibility.
- **Database system:** storage of datasets, models, monitoring runs, and results.
- **Dashboard framework:** visualization of drift results and performance metrics.

The selected stack is suitable for a bachelor thesis prototype because it supports both batch monitoring and streaming drift detection while remaining simple enough for demonstration and evaluation.

3.8 Chapter Summary

This chapter defined the requirements and architecture of the monitoring prototype. The system is designed around the comparison of reference and current data, with additional support for streaming drift detection and model performance monitoring. The next chapter describes how the proposed architecture is implemented in the prototype.

Bibliography

[1] S. R. Vadapalli. "Monitoring the Performance of Machine Learning Models in Production". In: *International Journal of Computer Trends and Technology* 70.9 (2022), pp. 38–42. DOI: 10.14445/22312803/IJCTT-V70I9P105.

[2] P. Koul and Y. Siddaramu. "Machine Learning in Production Engineering: A Comprehensive Review". In: *International Journal of Multidisciplinary Research in Arts, Science and Technology* 3.5 (2025), pp. 1–33. DOI: 10.61778/ijmrast.v3i5.131.

[3] O. Filippova and D. Maliugina. *Which Test Is the Best? We Compared 5 Methods to Detect Data Drift on Large Datasets*. [Online]. Available: https://www.evidentlyai.com/blog/data-drift-detection-large-datasets. Last updated: 2025-07-16. Accessed: 2026-05-19. 2022.

[4] The SciPy Community. *SciPy Documentation*. [Online]. Available: https://docs.scipy.org/doc/scipy/. Version 1.17.0. Accessed: 2026-05-19. 2026.

[5] J. Leest, I. Gerostathopoulos, P. Lago, and C. Raibulet. *Monitoring and Observability of Machine Learning Systems: Current Practices and Gaps*. 2025. arXiv: 2510.24142 [cs.SE].

[6] P. R. A. Puchakayala. "Model Monitoring Frameworks in Data Science and AI: Ensuring Robustness, Fairness, and Continuous Improvement". In: *International Journal of Computer Science and Engineering Research and Development* 13.2 (2023), pp. 40–59.

[7] M. Pavlovic. *Understanding Model Calibration: A Gentle Introduction and Visual Exploration of Calibration and the Expected Calibration Error (ECE)*. Accepted at ICLR Blogposts 2025. 2025. arXiv: 2501.19047 [stat.ME].

[8] R. Kakarla. "Autonomous Observability: ML-Optimized SLO/SLA Enforcement". In: *Computer Fraud and Security* 2024.5 (2024), pp. 75–84.

[9] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera. "A Unifying View on Dataset Shift in Classification". In: *Pattern Recognition* 45.1 (2012), pp. 521–530. DOI: 10.1016/j.patcog.2011.06.019.

[10] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia. "A Survey on Concept Drift Adaptation". In: *ACM Computing Surveys* 46.4 (2014), 44:1–44:37. DOI: 10.1145/2523813.

[11] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang. "Learning under Concept Drift: A Review". In: *IEEE Transactions on Knowledge and Data Engineering* 31.12 (2019), pp. 2346–2363. DOI: 10.1109/TKDE.2018.2876857.

[12] River Developers. *DDM: Drift Detection Method*. [Online]. Available: https://riverml.xyz/0.7.0/api/drift/DDM/. Accessed: 2026-05-19. 2026.

[13] River Developers. *ADWIN: Adaptive Windowing Method for Concept Drift Detection*. [Online]. Available: https://riverml.xyz/dev/api/drift/ADWIN/. Accessed: 2026-05-19. 2026.

[14] P. M. Gonçalves Jr., S. G. T. d. C. Santos, R. S. M. Barros, and D. C. L. Vieira. "A Comparative Study on Concept Drift Detectors". In: *Expert Systems with Applications* (2014). Preprint submitted to Elsevier, pp. 1–15.

[15] A. Balasubramanian. "End-to-End Model Lifecycle Management: An MLOPS Framework for Drift Detection, Root Cause Analysis, and Continuous Retraining". In: *International Journal of Multidisciplinary Research and Growth Evaluation* 1.1 (2020), pp. 92–102. DOI: 10.54660/.IJMRGE.2020.1.1-92-102.

[16] D. O. Hanchuk and S. O. Semerikov. "Implementing MLOps Practices for Effective Machine Learning Model Deployment: A Meta Synthesis". In: *CEUR Workshop Proceedings*. 2025, pp. 329–337. URL: https://ceur-ws.org/Vol-3918/paper404.pdf.

[17] M. E. Jang, A. Georgiadis, Y. Zhao, and F. Silavong. "DriftWatch: A Tool that Automatically Detects Data Drift and Extracts Representative Examples Affected by Drift". In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 6: Industry Track*. 2024, pp. 335–346. DOI: 10.18653/v1/2024.naacl-industry.28.

[18] H. M. Wong. *Model Monitoring, Data Drift Detection, and Efficient Model Retraining: A Review*. University of Wollongong Malaysia. Review paper. 2025.

[19] Aporia. *Machine Learning Model Monitoring 101*. [Online]. Available: https://www.aporia.com/machine-learning-model-monitoring-101/. 2022.

[20] M. Rake Linggar A. *Monitoring Model Performance*. [Online]. Available: https://towardsdatascience.com/monitoring-model-performance-51635c044f52/. Accessed: 2026-05-17. 2022.